

Universidad Rafael Landívar de Quetzaltenango
Facultad de Ingeniería.
Estructura de Datos II
Docente: Ing. Manuel Gustavo Rojas

Manual Técnico:
Tabla de posiciones de jugadores del mundial
Proyecto No.01

De León González, Alejandro Daniel
Carné: 1502425

Quetzaltenango, 08 de septiembre de 2026

1. Introducción

Este documento presenta el manual técnico del sistema "Tabla de Posiciones de Jugadores del Mundial", desarrollado en C# como Proyecto de Laboratorio No. 1 del curso Estructura de Datos II. El sistema permite registrar jugadores, actualizar sus estadísticas conforme avanza el torneo, buscar información individual, y generar reportes de Top 5 y listados generales ordenados por distintas categorías de desempeño.

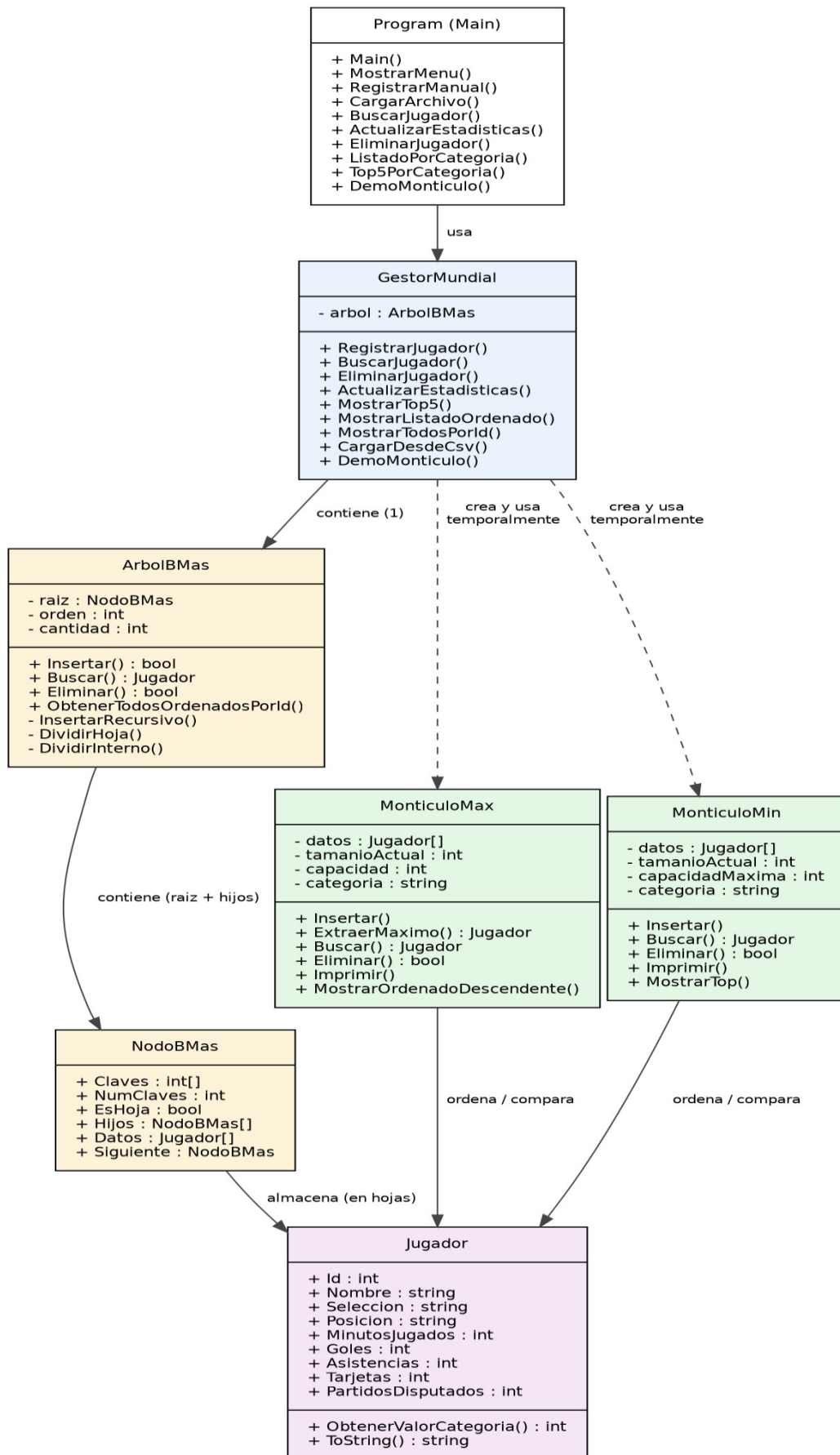
2. Descripción general del sistema

Cada jugador se representa mediante la clase Jugador, que almacena un identificador único (Id), su nombre, la selección a la que pertenece, su posición, y las estadísticas acumuladas: minutos jugados, goles, asistencias, tarjetas recibidas y partidos disputados. El Id de cada jugador funciona como la clave de indexación dentro del Árbol B+.

El sistema ofrece un menú de consola con nueve operaciones: registrar jugador manualmente, cargar jugadores desde un archivo CSV, buscar un jugador por Id, actualizar sus estadísticas, eliminarlo, mostrar el listado general ordenado por Id, mostrar el listado general ordenado por una categoría elegida por el usuario, mostrar el Top 5 de una categoría, y una demostración de las operaciones internas de los montículos (buscar, eliminar e imprimir)

3. Diagrama de clases

El siguiente diagrama muestra las clases que conforman el sistema y la relación entre ellas. GestorMundial actúa como fachada: es la única clase que Program conoce directamente, y es quien coordina al Árbol B+ (estructura permanente del catálogo) y a los montículos (estructuras temporales que se construyen bajo demanda para generar reportes).

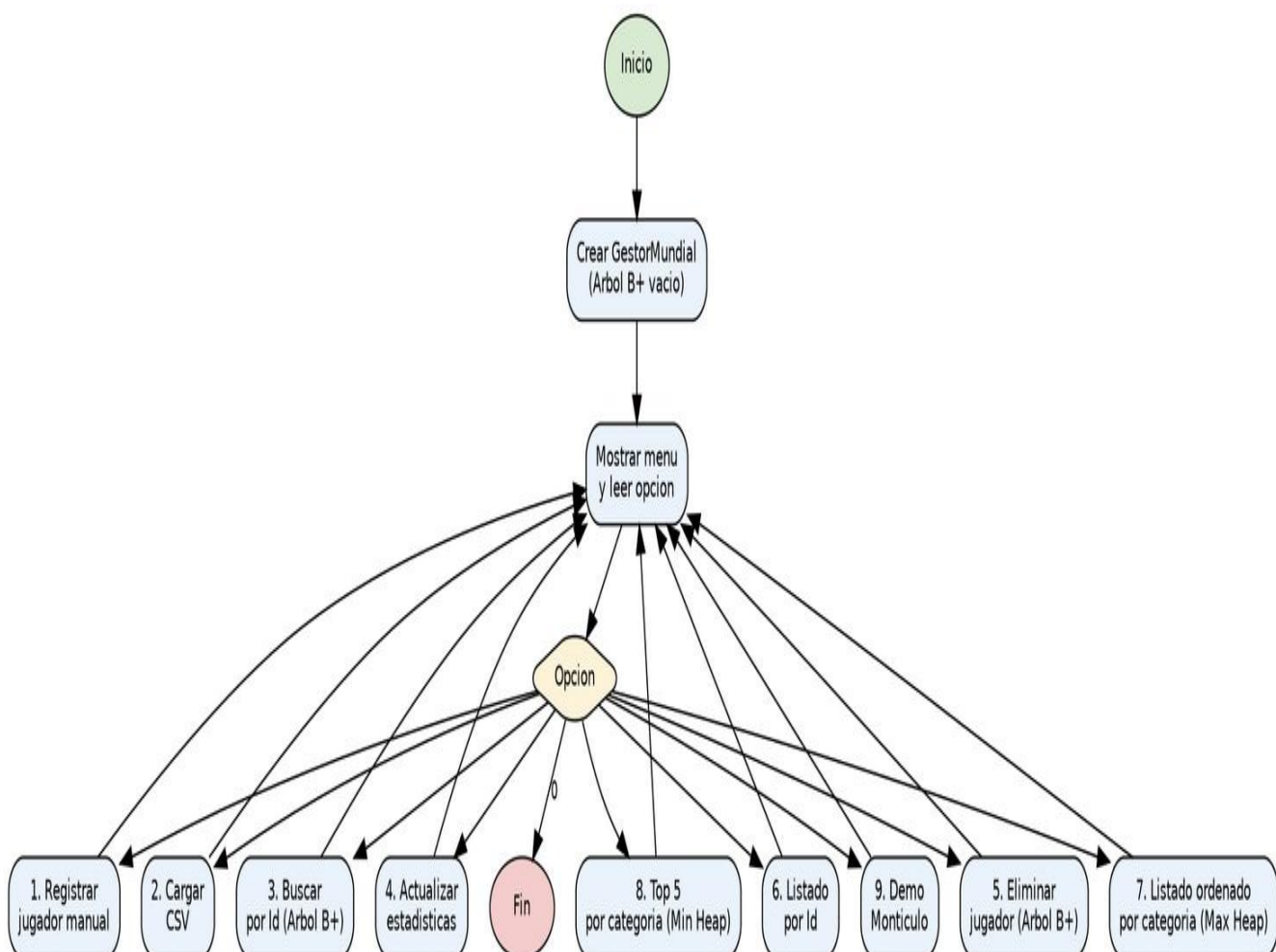


Relaciones principales:

- GestorMundial contiene una única instancia de ArbolBMas: es la estructura permanente donde vive el catálogo completo de jugadores mientras el programa se ejecuta.
- ArbolBMas está compuesto internamente por nodos NodoBMas, que pueden ser internos (guían la búsqueda) u hojas (almacenan a los Jugador reales y están enlazadas entre sí mediante el puntero Siguiente).
- GestorMundial crea instancias de MonticuloMax y MonticuloMin únicamente cuando el usuario solicita un reporte (Top 5 o listado ordenado por categoría); no son estructuras persistentes, se descartan después de imprimir el resultado.
- Tanto los montículos como las hojas del árbol trabajan sobre referencias a los mismos objetos Jugador, evitando duplicar la información en memoria.

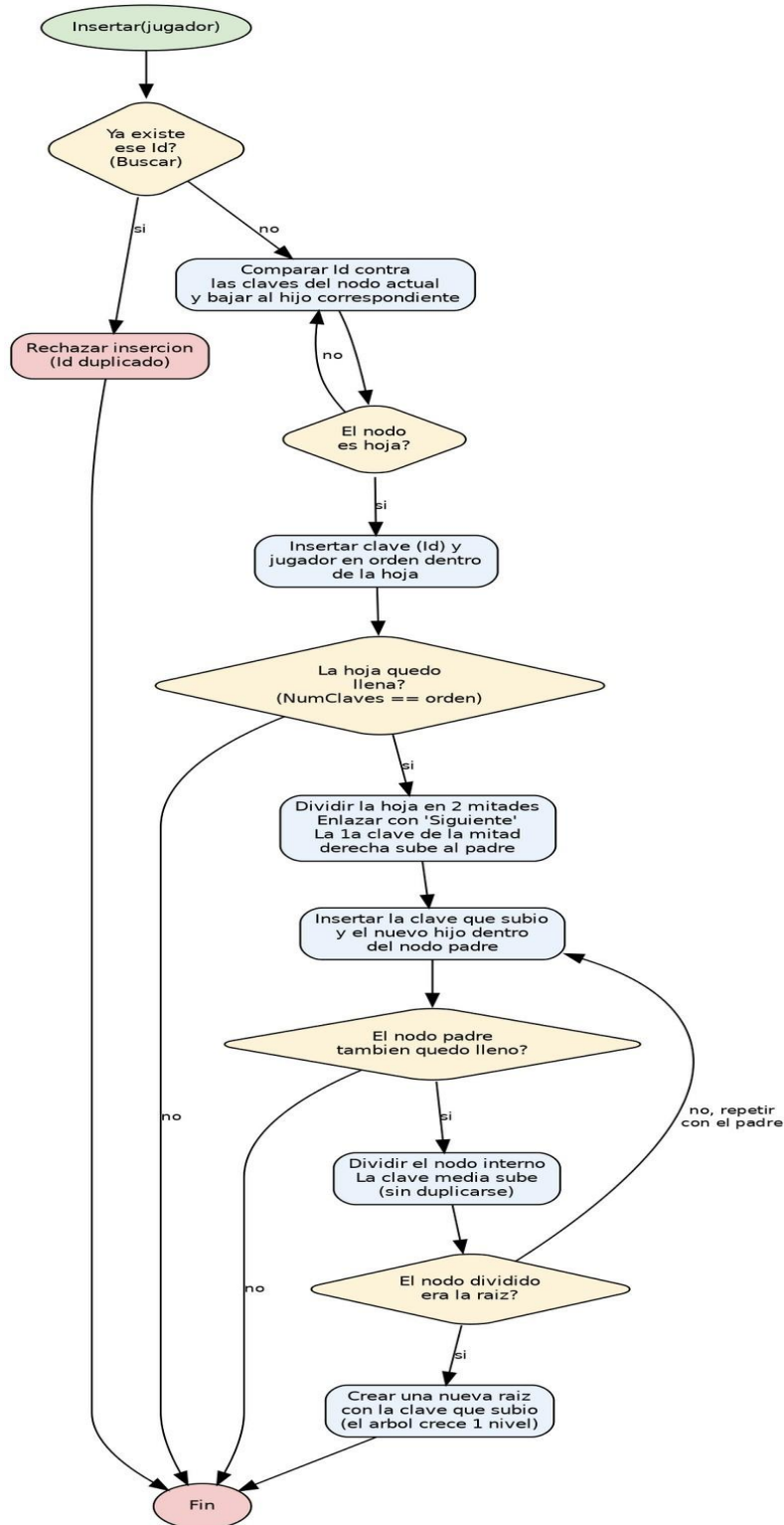
4. Diagrama de flujo general del programa

El programa funciona bajo un ciclo de menú: se muestra la lista de opciones, se lee la selección del usuario, se ejecuta la operación correspondiente delegándola a GestorMundial, y se vuelve a mostrar el menú hasta que el usuario elige salir (opción 0).



5. Diagrama de flujo: inserción en el Árbol B+

La operación más compleja del sistema es la inserción en el Árbol B+, porque puede provocar divisiones (splits) en cascada. El siguiente diagrama detalla su lógica, implementada de forma recursiva en el método InsertarRecursivo:



Puntos clave del algoritmo:

- Antes de insertar, se verifica que el Id no exista ya en el árbol (Buscar), para no permitir duplicados.
- Al dividir una hoja, la primera clave de la mitad derecha sube al padre y también permanece en la hoja (se duplica), porque en un Árbol B+ los datos reales solo viven en las hojas.
- Al dividir un nodo interno, la clave media sube al padre pero NO se duplica, porque los nodos internos solo sirven de guía para la búsqueda, no almacenan datos.
- Si el nodo que se divide es la raíz, se crea una nueva raíz y el árbol crece una unidad en altura; este es el único caso en que la altura del árbol aumenta

6. Justificación de la elección de cada estructura

A continuación se justifica la elección de cada estructura de datos según las operaciones más frecuentes del problema: búsqueda, inserción, eliminación, acceso y recorrido.

Estructura	Uso en el sistema	Por qué se eligió
Árbol B+	Índice principal del catálogo de jugadores, por Id.	Las hojas enlazadas permiten recorrer todo el catálogo en orden con una sola pasada secuencial, sin recursión. La altura logarítmica garantiza búsqueda, inserción y eliminación eficientes incluso con miles de jugadores.
Min Heap (acotado a 5)	Calcular el Top 5 de una categoría (goles, asistencias, minutos, etc.).	Mantener solo los 5 mejores durante el recorrido cuesta $O(n \log 5)$ en vez de ordenar todo el catálogo con $O(n \log n)$. La raíz siempre es el 'peor de los mejores', lo que permite decidir en $O(\log 5)$ si un nuevo jugador entra al Top 5.
Max Heap	Generar el listado completo ordenado de mayor a menor por una categoría.	Permite obtener el máximo actual en $O(1)$ y producir un listado totalmente ordenado mediante extracciones sucesivas (heap sort), sin necesitar una estructura de comparación externa como las que provee .NET.

Se descartó usar una lista enlazada simple como estructura principal porque la búsqueda por Id sería $O(n)$; también se descartó un Árbol Binario de Búsqueda (BST) simple porque, sin balanceo, su rendimiento se degrada a $O(n)$ en el peor caso (por ejemplo, si los Id se insertan en orden ascendente). El Árbol B+ resuelve ambos problemas manteniéndose balanceado por construcción.

7. Complejidad algorítmica

Operación	Árbol B+	Min Heap (k=5)	Max Heap
Insertar	$O(\log n)$	$O(\log k)$	$O(\log n)$
Buscar	$O(\log n)$	$O(k)$ — lineal	$O(n)$ — lineal
Eliminar	$O(\log n)^*$	$O(k)$ — lineal	$O(n)$ — lineal
Recorrer / listar todo	$O(n)$	$O(k \log k)$	$O(n \log n)$

8. Explicación de la implementación por clase

Jugador.cs

Clase modelo que representa a un jugador y sus estadísticas. Incluye el método `ObtenerValorCategoria(string categoria)`, que traduce el nombre de una categoría ("goles", "asistencias", "minutos", "tarjetas", "partidos") al valor numérico correspondiente; este método es el que usan ambos montículos para comparar jugadores sin necesidad de escribir un comparador distinto por cada categoría.

NodoBMas.cs

Representa un nodo del Árbol B+. Contiene arreglos de tamaño fijo para las claves, los hijos y los datos, más una bandera `EsHoja` y un puntero `Siguiente` que enlaza las hojas entre sí para el recorrido secuencial.

ArbolBMas.cs

Implementa el Árbol B+ de orden 4 (máximo 3 claves por nodo). Contiene la lógica recursiva de inserción con manejo de divisiones (`DividirHoja` y `DividirInterno`), búsqueda descendente por comparación de claves, eliminación directa sobre la hoja, y el recorrido secuencial `ObtenerTodosOrdenadosPorId` que aprovecha el enlace entre hojas.

MonticuloMax.cs y MonticuloMin.cs

Implementan un heap binario sobre un arreglo propio, con redimensionamiento manual (duplicando la capacidad) en el caso del Max Heap, y con capacidad fija k en el caso del Min Heap acotado. Ambas clases exponen `Insertar`, `Buscar`, `Eliminar` e `Imprimir`, además de su operación de reporte (`ExtraerMaximo/MostrarOrdenadoDescendente` para el Max Heap, `MostrarTop` para el Min Heap).

GestorMundial.cs

Actúa como fachada entre el menú (`Program.cs`) y las estructuras de datos. Centraliza la carga de archivos CSV con manejo de errores (líneas incompletas, formato inválido, Id duplicado) y coordina la construcción temporal de los montículos a partir del contenido del Árbol B+ para generar los reportes.

Program.cs

Contiene el punto de entrada (Main) y el ciclo de menú. Cada opción del menú delega el trabajo real a un método de GestorMundial; Program solo se encarga de leer la entrada del usuario y mostrar los resultados.

9. Limitaciones conocidas

- La eliminación en el Árbol B+ no fusiona ni redistribuye nodos con pocas claves tras la eliminación
- El orden del Árbol B+ está fijo en 4 mediante una constante en el constructor de GestorMundial; para catálogos muy grandes convendría exponerlo como parámetro configurable.
- La búsqueda y eliminación dentro de los montículos son lineales ($O(n)$ o $O(k)$), porque un heap binario solo garantiza el orden entre padre e hijo, no un orden total como el que sí ofrece el Árbol B+.